

COMP202 Week 4 - solutions

Note: As you might suspect, most of these exercises can be solved essentially using variations of efficient sorting algorithms (e.g., MergeSort or HeapSort or (randomized) QuickSort).

1. Suppose that S is a set of n integers. Describe an efficient algorithm to determine whether there are two (or more) equal integers in S . What is the running time of your algorithm?

(Note: This can easily be done in time $O(n^2)$. (How?) Can you do better?)

Solution: We start performing the MergeSort or HeapSort or randomized Quicksort algorithm.

Suppose we're using the MergeSort algorithm. Then, as we're merging the lists (or some of the sublists in one of the recursive calls), we merely check to see if the two items we're comparing as we merge the lists are equal to each other. If so, we can stop and report "Yes". If we never find an equal pair of integers, then when we're finished with the sorting, we report "No".

For QuickSort, observe that it does not do "merging" of sublists, but just "appends" the (recursively sorted) L (element strictly smaller than pivot) on left to E (elements equal to pivot) in the middle and appends that to (recursively sorted) R (element strictly larger than pivot) on the right.

The solution with the randomized QuickSort algorithm is similar to MergeSort, but when we're creating the sublists using the pivot element, if the sublist we called E is ever non-empty at some step, then we again report "Yes". More precisely: if in any level of recursion the set E has more than one element, then we output "Yes" (I assume that at the bottom of recursion arrays have length 1), and we output "No" if every set E has only one element.

Using the first method takes time $O(n \log n)$ in the worst case as we are basically performing the MergeSort algorithm. The second method takes expected time $O(n \log n)$ using the randomized QuickSort method.

If you want to use HeapSort, this is also fine. Simply maintain a variable that stores the last number removed by the RemoveMax operation. Note that a duplicate in a set means in sorted order there must be 2 same values occurring next to each other. All you need to check now is whether during the execution of HeapSort there's any RemoveMax operation that returns the same value of the previous RemoveMax operation. This only adds an overhead of $O(1)$ per RemoveMax operation, so the total worst-case running time of this procedure is also $O(n \log n)$.

2. Suppose you are given two sequences X and Y of k integers each, possibly containing duplicates. Describe an efficient algorithm to determine if X and Y contain the same set of elements (possibly in different orders). What is the running time of your procedure?

(Note: There are two ways this could be interpreted. Suppose that $X = (1, 1, 1, 1, 2)$ and $Y = (2, 2, 1, 2, 1)$. If we take only the collection of distinct elements into account, then since X and Y both contain 1 and 2 (only "counting" a duplicated item once), we might consider them to "contain the same set of elements". If, on the other hand, we want to consider both the elements *and* the number of occurrences, then X and Y do not contain the same set of elements, as the number of occurrences of 1 and 2 in the lists are clearly different.

Consider both of these cases in your analysis.)

Solution: Surprise, surprise... Apply the MergeSort algorithm on each list X and Y separately. Then when we're done with sorting each list, we compare them element by element to see if they are equal.

The only difference between the two interpretations is how we might consider merging the sublists. If we want to exclude the duplicates to consider X and Y equal, then we need only include one copy when we merge the lists. In other words, if we have $X = (1, 1, 1, 1, 2)$ and $Y = (2, 2, 1, 2, 1)$, when we sort and "merge" the lists, we can end up with the new lists $X' = (1, 2)$ and $Y' = (1, 2)$ which we compare and see they are equal.

If we wish to include the duplicates, then when we sort the lists we will end up with $(1, 1, 1, 1, 2)$ and $(1, 1, 2, 2, 2)$ which we then compare element by element and see they are different lists.

In any case, depending upon how we wish to compare the lists, we can still do this in time $O(n \log n)$, as we apply MergeSort twice (or some small variation of it if we remove the duplicates).

3. Let A and B be two sequences, each having k integers. Given an integer x , describe an $O(k \log k)$ algorithm to determine if there is an integer a in A and an integer b in B such that $x = a + b$.

Solution I: We will first show a non-recursive solution. First using any sorting algorithm (e.g., MergeSort, HeapSort), sort one of these tables, say B , obtaining a sorted (in non-decreasing order) B . The total time is $O(k \log k)$. Then take sequentially each element a from table A and check by binary search if number $x - a$ is present in B ; this takes time $O(\log k)$. If the answer is yes then we can stop, otherwise we continue. The total time of this part is the number of elements in A times $O(\log k)$, that is, $O(k \log k)$.

Solution II: The idea here is that we'll apply the (randomized) QuickSort procedure on the set A , while at the same time searching for the solution that we want.

What's the idea? Pick a pivot element for A , and divide A into the three sets L_A, R_A, E_A as normal. Suppose the value of this pivot element is k_1 . Since we're given the value x , use $x - k_1$ as the pivot element for B . As we subdivide B into the sets L_B, R_B, E_B , if the set E_B is non-empty, we can stop and we're done with a positive answer. This is because k_1 is in A and we would have just shown that the value $x - k_1$ is in B .

Remark: Note that it is important here to choose the pivot k_1 randomly from A , because only then $x - k_1$ will be (random) pivot element for B , and so it will divide B onto two roughly same size subarrays with probability at least $1/2$ (see the analysis of QuickSort).

Otherwise we can continue into the next recursive function calls. We would pick a pivot value for the subset L_A . Say this value is k_2 . We would then use the value $x - k_2$ as a pivot value for the set R_B . Why? Because, once again, as we subdivide the set R_B we will determine if there is element having value $x - k_2$ in it. If so, we combine this with k_2 to find our solution to $x = a + b$ as desired.

Remark: As a recursive subproblem we combine L_A here with R_B for the following reason. We know that each element q in L_A is $q < k_1$ and each element p in L_B is $p < x - k_1$, therefore $q + p < x$, so L_A cannot be combined with L_B , but possibly only with R_B . Symmetrically, we also combine, as the second smaller subproblem for the recursion, R_A and L_B .

So we can continue in this fashion to determine if there is solution of the form we're looking for. Note that at some stages we might be able to further short-circuit this search process. If, for example, during the first steps, we find that the set R_B (consisting of items that are greater

than $x - k_1$) is empty, then we need not consider any elements in L_A . (Why?)

Again, since we're doing the randomized Quicksort algorithm, this whole procedure will run in (expected) time $O(n \log n)$.

4. You are given an array A of n integers, where each element lies in the range $\{1, 2, \dots, k\}$.

Describe an algorithm that runs in time $O(n+k)$ to determine whether any value appears more than $n/3$ times in A .

Solution: We don't use Counting Sort directly, but we use a subroutine that it employs. Here is the algorithm.

- Create an array $C[1..k]$ and initialise all entries to zero.
- Scan the array A . During the scan, for each element $a \in A$, increment $C[a]$.
- Scan the array C . If any entry is strictly greater than $n/3$, output "Yes"; otherwise output "No".

A sketch of correctness: For each value $i \in \{1, \dots, k\}$, the array C stores exactly the number of times i appears in A . The algorithm therefore outputs "Yes" if and only if there exists a value that appears more than $n/3$ times in A .

The first scan of A takes $O(n)$ time. The scan of C takes $O(k)$ time. Hence the total running time is $O(n + k)$.